

Application Services Model

2026 Sebastián Samaruga. <https://sebxama.blogspot.com>

This work is licensed under a [Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License](#).

Data Model:

Reference Model (ISO TopicMaps TMRM):

(Key, Value) Underlying Unified Graphs Representation.

Sets Based Model Representation:

Subjects Set.

Predicates Set.

Objects Set.

Subject Kinds Set (Predicates Set intersection with Objects Set).

Predicate Kinds Set (Subjects Set intersection with Objects Set).

Object Kinds Set (Subjects Set intersection with Predicates Set).

Contexts Set (Subjects Set, Predicates Set and Objects Set intersection).

Model Classes:

Resources:

Resource

- URI : String

- primelD : BigInteger
- occurrences : Set<Occurrence>

Occurrences:

Occurrence:

- context : Statement
- resource : Resource
- kind : Kind

Occurrence Instance Statements:
(Occurrence, Statement, Resource, Kind);

ContextOccurrence : Occurrence

SubjectOccurrence : Occurrence

PredicateOccurrence : Occurrence

ObjectOccurrence : Occurrence

Statement : ContextOccurrence, SubjectOccurrence, PredicateOccurrence,
ObjectOccurrence

Types / Roles / Kinds:

Given a Statement Subject, its Attributes and Values are the Statement Predicate and Object, respectively.

Given a Statement Predicate, its Attributes and Values are the Statement Subject and Object, respectively.

Given a Statement Object, its Attributes and Values are the Statement Predicate and Subject, respectively.

Kind<Occurrence, Attribute, Value> : Occurrence

- occurrences : Set<Occurrence>
- attributes : Set<Attribute>
- values : Map<Attribute, Set<Value>>

Kind Instances Statements:
(Kind, Occurrence, Attribute, Value);

SubjectKind : Kind<SubjectOccurrence, PredicateOccurrence, ObjectOccurrence>, SubjectOccurrence

PredicateKind : Kind<PredicateOccurrence, SubjectOccurrence, ObjectOccurrence>, PredicateOccurrence

ObjectKind : Kind<ObjectOccurrence, PredicateOccurrence, SubjectOccurrence>, ObjectOccurrence

DOM / DCI (MDA) Design Pattern Model Classes:

TODO

RDF Statements (Quads) Data Model Encoding:

Canonical CSPO Interpretation:

(Context, Object, Attribute, Value);

Resource Statements:

(Resource, Occurrence, Attribute, Value);

Occurrence Statements:

(Occurrence, Statement, Resource, Kind);

Types / Roles / Kinds Statements:

(Kind, Occurrence, Attribute, Value)

Schema Statements (inferred from Kind Statements):

(Type, Type, Attribute, Type)

Instance Statements:

(Instance, Type, Attribute, Value);

Context Behavior Schema Statements:

(VerbInfinitive, DomainActorRole, Verb, RangeActorRole):

Primitive Verbs:

NodeCreation
NodeDeletion
EdgeCreation
EdgeDeletion
ValueAssignment
ValueTransform
EventEmmision

Interaction Behavior Instance Statements:

(VerbSubstantive, DomainActorInstance, Action, DomainActorInstance);

Primitive Executions:

NodeCreated
NodeDeleted
EdgeCreated
EdgeDeleted
ValueAssigned
ValueTransformed
EventEmited

DOM / DCI Design Patterns Statements Encoding:

Schema, Instance, Context and Interaction Statements serialize the DOM / DCI Design Pattern Model Classes.

Leveraging Data Statements inference as an Schema Statements as Grammar Possible Statements Productions:

Infer Schema (Grammar) from Data (Productions / Terminals):

FCA Contexts Instance Statements:

Nested Contexts (Enables Dimensional features):
(Context, Object, Context, Attribute);

Algebraic Semantic Embeddings (CPPE):

Prime Numbers Identifiers:

Every Resource is assigned a unique Prime Number ID (sequence) associated with its URI (via Registry Helper Service API).

Leveraging Statements Occurrences Metadata with FCA:

Interpretation of RDF Statements as FCA Contexts:

Subjects as FCA Contexts: Statements Objects are FCA Context Objects, Statement Predicates are FCA Context Attributes.

Predicates as FCA Contexts: Statement Subjects are FCA Context Objects, Statement Objects are FCA Context Attributes.

Objects as FCA Contexts: Statement Subjects are FCA Context Objects, Statement Predicates are FCA Context Attributes.

FCA Contexts serialization as RDF Graph Quads Statements.

CPPE. FCA-based Embeddings: A Deterministic Approach:

Resource Embeddings are calculated by the product of the Resource's Prime ID by the product of the Resource's Occurrences Attributes and Values Prime IDs.

We will replace LLM-based embeddings with deterministic, structural embeddings derived from FCA contexts and prime number products. This provides explainable similarity based on shared roles and relationships.

- **Contextual Prime Product Embedding (CPPE):** For any Occurrence (i.e., a resource in a specific statement), we can calculate an embedding based on its relational context.
 1. **Define FCA Contexts:** For a given relation (predicate), we can form an FCA

context. Example: For the predicate :worksFor:

- **Objects (G):** The set of all subjects of :worksFor statements (e.g., {id:Alice, id:Bob}).
- **Attributes (M):** The set of all objects of :worksFor statements (e.g., {id:Google, id:StartupX}).
- 2. **Calculate Prime Product:** The CPPE for id:Google within the :worksFor context is the product of the primeIDs of all employees who work there. $CPPE(Google, worksFor) = primeID(Alice) * primeID(Bob) * \dots$
- **Similarity Calculation & Inference:**
 - **Similarity:** The similarity between two entities in the same context is the Greatest Common Divisor (GCD) of their CPPEs. $GCD(CPPE(Google), CPPE(StartupX))$ reveals the primeID product of their shared employees, giving a measure of personnel overlap.
 - **Relational Inference:** We can infer complex relationships. Consider the goal of finding an "uncle".
 1. Calculate the CPPE for "Person A" in the :brotherOf context (the product of their siblings' primes).
 2. Calculate the CPPE for "Person B" in the :fatherOf context (the product of their children's primes).
 3. If $GCD(CPPE_brotherOf(A), CPPE_fatherOf(B)) > 1$, it means A is the brother of B's father. The system can then materialize a new triple: (A, :uncleOf, ChildOfB). This inference is stored and queryable.

FCA-based Relational Schema Inference:

The system can infer relational schemas (rules or "upper concepts") from the structure of the data itself using FCA.

- **FCA Contexts for Relational Analysis:** We use three types of FCA contexts to analyze relationships from different perspectives:
 1. **Predicate-as-Context:** (FCA Objects: Statement Subjects, FCA Attributes: Statement Objects, FCA Context: Statement Predicate). This context reveals which types of subjects relate to which types of objects for a given predicate.
 2. **Subject-as-Context:** (FCA Objects: Statement Predicates, FCA Attributes: Statement Objects, FCA Context: Statement Subjects). This reveals all the relationships and objects associated with a given subject, defining its role.
 3. **Object-as-Context:** (FCA Objects: Statement Subjects, FCA Attributes: Statements Predicates, FCA Context: Statement Objects). This reveals all the subjects and actions that affect a given object.
- **Algorithm: Inferring Relational Schema:**
 1. **Select Context:** For a given predicate P (e.g., :worksOn), the Alignment Service constructs the Predicate-as-Context.
 2. **Build Lattice:** It uses an FCA library (e.g., fcalib) to compute the concept lattice

from this context.

3. **Identify Formal Concepts:** Each node in the lattice is a formal concept (A, B), where A is a set of subjects (the "extent") and B is the set of objects they all share (the "intent").
4. **Materialize Schema:** Each formal concept represents an inferred relational schema or "upper concept". The system creates a new RDF class for this concept. For a concept where the extent is {dev1, dev2} (both :Developers) and the intent is {projA, projB} (both :Projects), the system can materialize a schema:
:DeveloperWorksOnProject a rdfs:Class, :RelationalSchema ;
:hasDomain :Developer ;
:hasRange :Project .

Relational Context Vectors

The core of this approach is the Relational Context Vector (RCV). For any given statement (a reified triple), we compute a vector of three BigInteger values, (S, P, O). Each component is a CPPE calculated from one of the three FCA context perspectives, providing a holistic numerical signature of the statement's role in the graph.

- **RCV Definition:** $RCV(statement) = (S, P, O)$
 - **S (Subject Context Embedding):** The CPPE of the statement's subject from the Subject-as-Context perspective. This number encodes everything the subject does. $S = calculateCPPE(statement.subject, SubjectAsContext)$
 - **P (Predicate Context Embedding):** The CPPE of the statement's predicate from the Predicate-as-Context perspective. This number encodes every subject-object pair the predicate connects. $P = calculateCPPE(statement.predicate, PredicateAsContext)$
 - **O (Object Context Embedding):** The CPPE of the statement's object from the Object-as-Context perspective. This number encodes everything that happens to the object. $O = calculateCPPE(statement.object, ObjectAsContext)$
- **Implementation:** A Java record $RCV(BigInteger s, BigInteger p, BigInteger o)$. The Index Service is responsible for calculating and caching the RCV for every reified statement in the graph.

Schema Archetypes:

This dual representation is key to performing inference.

- **Instance RCV:** The RCV calculated for a specific, concrete statement (e.g., `stmt_123: (dev:Alice, :worksOn, proj:Orion)`) is its unique numerical signature. It represents a single data point.
- **Schema RCV (Archetype):** The RCV for a relational schema (e.g., the

:DeveloperWorksOnProject schema) is an "archetype" vector. It is calculated by finding the Least Common Multiple (LCM) of the corresponding components of all instance RCVs that belong to that schema.

- **Algorithm: calculateSchemaRCV(schemaURI)**
 1. Find all instance statements s_i where $s_i \text{ rdf:type schemaURI}$.
 2. For each instance s_i , retrieve its cached $RCV_i = (S_i, P_i, O_i)$.
 3. Calculate the schema RCV components:
 - $S_{\text{schema}} = \text{LCM}(S_1, S_2, \dots, S_n)$
 - $P_{\text{schema}} = \text{LCM}(P_1, P_2, \dots, P_n)$
 - $O_{\text{schema}} = \text{LCM}(O_1, O_2, \dots, O_n)$
 4. The result $(S_{\text{schema}}, P_{\text{schema}}, O_{\text{schema}})$ is the numerical archetype for the schema. The LCM ensures that the schema's numerical signature is "divisible" by all of its instances.

Subsumption:

Subsumption / Instance Checking (rdf:type):

- **Concept:** An instance belongs to a schema if the instance's RCV "divides into" the schema's RCV.
- **Algorithm: isInstanceOf(instanceRCV, schemaRCV)**
 1. Perform a component-wise modulo operation.
 2. $\text{boolean isS} = \text{schemaRCV.s.mod(instanceRCV.s.equals(BigInteger.ZERO))};$
 3. $\text{boolean isP} = \text{schemaRCV.p.mod(instanceRCV.p.equals(BigInteger.ZERO))};$
 4. $\text{boolean isO} = \text{schemaRCV.o.mod(instanceRCV.o.equals(BigInteger.ZERO))};$
 5. Return $\text{isS} \ \&\& \ \text{isP} \ \&\& \ \text{isO}$.
- **Use Case:** This is a high-speed, purely numerical method for checking type constraints, which can be performed in memory without a complex graph query.

Property Chains

Define composed relations (e.g., knowsLanguage) and leverage reasoners for closure. This section details the specific numerical algorithm for the $(\text{:Developer})\text{--}[\text{worksOn}]\text{--}(\text{:Project})$ and $(\text{:Project})\text{--}[\text{usesLanguage}]\text{--}(\text{:Language}) \implies (\text{:Developer})\text{--}[\text{knowsLanguage}]\text{--}(\text{:Language})$ inference.

- Step 1: Define the Composition Operator
 $\text{compose}(RCV1, RCV2)$
 - **Inferred Subject (S_{inferred}):** $S_{\text{inferred}} = RCV1.s * RCV2.o$
 - **Inferred Object (O_{inferred}):** $O_{\text{inferred}} = RCV1.s * RCV2.o$
 - **Inferred Predicate (P_{inferred}):** $P_{\text{inferred}} = RCV1.p * RCV2.p$
- **Step 2: Calculate Schema Archetypes**
 - The Index Service calculates archetypal RCVs for source schemas:

- $RCV_worksOn_schema = (S_wo, P_wo, O_wo)$
 - $RCV_usesLang_schema = (S_ul, P_ul, O_ul)$
- It then calculates the archetypal RCV for the inferred schema (knowsLanguage):
 - $S_kl = S_wo * O_ul$
 - $P_kl = P_wo * P_ul$
 - $O_kl = S_wo * O_ul$
- This resulting $RCV_knowsLang_schema = (S_kl, P_kl, O_kl)$ is stored.
- **Step 3: The Inference Algorithm at Query Time**
 A user asks: "Does dev:Alice know lang:Java?"
 1. **Retrieve Instance RCVs:** Retrieve RCV1 for (dev:Alice, :worksOn, proj:Orion) and RCV2 for (proj:Orion, :usesLanguage, lang:Java).
 2. **Calculate Hypothetical Instance RCV:** $RCV_hypothetical = compose(RCV1, RCV2)$.
 3. **Retrieve Schema Archetype:** Retrieve $RCV_knowsLang_schema$.
 4. **Perform Numerical Check:** $boolean\ knows = isInstanceOf(RCV_hypothetical, RCV_knowsLang_schema)$.
 5. **Result:** If knows is true, the inference is validated.

Querying and Traversal by Numerical Properties:

- **Find by Relational Role:** "Find all entities that have acted as a :Developer".
 - The query becomes: "Find all statements whose instanceRCV.s component divides $RCV_dev_schema.s$."
- **Traversal by Numerical Similarity:**
 - Start at $stmt_A$ with RCV_A .
 - The next step: "Find $stmt_B$ whose RCV_B has the highest GCD with RCV_A ."
 - Allows traversal based on numerically similar relational contexts.

Graph Pipeline Specification: CPPE Algebra for Pipeline Stream Actions:

The **Contextual Prime Product Embedding (CPPE)** algebra treats every node in the graph not as a string or a pointer, but as a unique prime number. By leveraging the **Fundamental Theorem of Arithmetic**, relationships and sets are represented as products. This allows the Reactive Stream Pipeline to perform complex graph "joins" and "path traversals" using simple CPU-native arithmetic.

1. Algebraic Definitions

Let $\mathcal{P}$ be the set of unique Prime IDs assigned to Resource URIs.

For any ContextPoint x :

- $\text{id}(x) \in \mathcal{P}$ (The singleton identity).
- $\text{emb}(x)$ (The cumulative embedding).

The Prime Product Rule:

The embedding of a triadic occurrence (c, o, a) —where c is Context, o is Object, and a is Attribute—is defined as the product of their respective recursive embeddings:

$$\text{emb}(c, o, a) = \text{emb}(c) \times \text{emb}(o) \times \text{emb}(a)$$

Because every factor is a prime (or a product of primes), the result is a unique "coordinate" in the integer space that encodes the entire lineage of the relationship.

2. Pipeline Stream Operators (The CPPE Monad)

In a reactive stream (e.g., `Flux<FCAContextPoint>`), the following algebraic operators are used for "Stream Action":

A. The "Contains" Operator (Sub-graph Filtering):

To check if a stream of Objects belongs to a certain Context C or possesses an Attribute A :

- **Logic:** $x \in \text{Context}(C) \iff \text{emb}(x) \pmod{\text{id}(C)} \equiv 0$
- **Reactive Implementation:**
// Filter objects that are part of the 'Accounting' context
`objectStream.filter(obj ->
obj.getPrimeIDEmbedding().remainder(accountingID).equals(BigInteger.ZERO));`

B. The "Intersection" Operator (Aggregation):

To find commonalities between two streams (finding the "Type" or "Concept"):

- **Logic:** $\text{Type}(x, y) = \text{gcd}(\text{emb}(x), \text{emb}(y))$
- **Stream Action:** The Greatest Common Divisor (GCD) of the embeddings of two nodes represents their shared FCA attributes/contexts.
- **Aggregation Use Case:** When the stream encounters multiple SPO triples, it calculates the "running GCD" of the attributes to infer the Schema (the most specific common super-type).

C. The "Axis Shift" Operator (Alignment):

Alignment predicts a value for a new context by calculating the "Ratio" of change between existing embeddings.

- **Logic:** $\text{Shift}(x, \Delta) = \text{emb}(x) \times \frac{\text{id}(\text{NewContext})}{\text{id}(\text{OldContext})}$
- **Stream Action:** By dividing by the "Old Axis" prime and multiplying by the "New Axis" prime, we re-project the object into a new coordinate.

3. Pipeline Stages Revisited via CPPE:

3.1 Aggregation (Algebraic Type Inference):

The Aggregation pipeline consumes raw SPO streams and produces **"Type Tensors"**.

1. **Input:** Stream of rotated triples (S, P, O) .
2. **Action:** For all x sharing attribute a , calculate the product $\prod \text{id}(x)$.
3. **Inference:** If $\text{emb}(x)$ is divisible by the product of a set of attribute primes, x is classified into that Type.
4. **Complexity:** $O(1)$ lookup via modulo instead of $O(N)$ graph search.

3.2 Alignment (Vector/Prime Space Prediction):

Alignment uses the **Prime Ratio** to detect structural similarity.

1. **Input:** Aggregated Types.
2. **Action:** Map the distance between objects O_1 and O_2 as $d = \frac{\text{emb}(O_1)}{\text{emb}(O_2)}$.

3. **Discovery:** If O_3 and O_4 share the same ratio d , the system aligns them as being "analogous" in different contexts (e.g., "CEO is to Company A what Principal is to School B").

3.3 Activation (State Transition Product):

Activation is the "Activation Energy" required to move from one prime state to another.

- **Transition Formula:** $\text{Transition} = \text{emb}(\text{Prev}) \otimes \text{emb}(\text{Next})$
- **Monadic Wrap:** The Occurrence Monad captures the prime state of an actor. When the stream processes a "Role" event, it multiplies the actor's prime ID by the role's prime ID.
- **Trigger:** If the resulting product matches a known "Success Pattern" (a pre-calculated product of requirements), the **Activation** is fired (e.g., triggering a webhook or a UI change).

Dimensional Features:

Enabled by RDF Quads FCA (Recursive) Contexts serialization.

Order Inference.

Comparisons given axes.

Dimensional aggregation / traversal.

Sequences (previous, current -> next state / values Inference).

TODO

Semantic Engine:

Datasources:

Anything (any backend / format) to RDF SPO Triples.

Synchronization features via Event Bus.

Augmentation Pipeline Services:

Aggregation Service:

Type Inference of Resource Occurrences.

Helper: Registry Service.

Alignment Service:

Link Prediction and Ontology Matching.

Helper: Naming Service.

Activation Service:

Behavior Contexts Schema Inference / Resolution.

Behavior Interactions Instances Inference / Resolution.

Helper: Index Service (resolve discrete role / actor values in contexts / interactions).

Runtime:

Process Graph State as an Executable Model.

TODO

Reactive Message Driven Event Bus Architecture:

Services communicate between a shared Events Topic of Graph CRUD events. When messages match its input signature / pattern they process them in a functional stream based approach, with corresponding aggregation, matching and activation functional behavior, and then emit (publishes) its service layer inferred output messages (Statements) for further processing.

Events / Topics (Graph CRUD Events):

Main Events Topic of Graph CRUD Events. Augmentation Services produces / consumes RDF Statements (Quads) Messages.

TODO

Message Formats:

RDF Statements (Quads).

Message Pattern Matching:

Each service listens to Topic Events messages, filtering those who match its input signatures / pattern.

Aggregation Service Pattern:

Consumes: Raw (Data) Statements

Produces: Occurrence Statements, Kind Statements

Alignment Service Pattern:

Consumes: Occurrence Statements, Kind Statements

Produces: Type Statements, Schema Statements

Activation Service Pattern:

Consumes: Type Statements, Schema Statements

Produces: Context Behavior, Interaction Behavior Statements

Message Routes / Blackboard:

Datasource -> Topic -> Aggregation

Aggregation -> Topic -> Alignment

Alignment -> Topic -> Activation

Activation -> Topic > Runtime

These are not “hardcoded” routes. A service’s outputs (Alignment Service outputs, for example) could be further consumed by the Aggregation Service for further inferences outputs. Datasource service could, in turn, process stream messages for backend synchronization.

Functional Stream Processing:

Each service has a functional stream pipeline endpoint for processing consumed message inputs which, leveraging helper services functional APIs, perform (functional monadic) transformations which lead to inference of its output messages.

TODO

W3C DIDs Distributed Resource Identifiers:

Enable Blockchain Features (Registry Helper Service).

TODO

Helper Functional Streams Services:

Leverages Augmentation Pipeline Services. Functional APIs for augmenting stream processing.

Registry:

URI / PrimeID / W3C DIDs Core Resources Registry API.

Criteria / Pattern Based Resource Resolution API.

Blockchain Features.

Type Inference. Classification (Aggregation).

TODO

Naming:

Key / Value Store Features API (Graph GenAI Context).

TMRM ISO TopicMaps Reference Model (Key / Value) Features API.

Graph GenAI Features.

Link / Attribute inference, Ontology Matching Features (Alignment).

TODO

Index:

FCA Features API Implementation.

Semantic Embeddings.

CPPE Features API Implementation.

Sets Based Representation API.

Context State Transitions. Regression (Activation). Resolve discrete role / actor values in contexts / interactions.

TODO